

Quotient-Space Based Learning for Motion Planning Infeasibility

Alamanda Hasitha (2023102072)

Public Git Repo Link: [Click here for the link](#)

Abstract

Determining whether a robot arm can move between two configurations in the presence of obstacles is a central problem in motion planning. Classical planners such as RRT are reliable but computationally expensive, making repeated queries impractical in time-sensitive settings. This project presents a learning-based surrogate that answers the feasibility question in a single forward pass of a neural network. We frame the problem as a five-class classification task using a *quotient-space decomposition*: rather than predicting only feasible or infeasible, the model also identifies which sub-chain of the arm first becomes infeasible, providing actionable diagnostic information. A Quotient-Space Transformer (QST) is trained on 100,000 labelled samples generated via PyBullet and RRT. The model achieves **98.68%** overall validation accuracy with per-class accuracy exceeding 97% on four of five classes, demonstrating that geometric feasibility can be reliably inferred directly from raw obstacle coordinates and robot configuration angles without explicit collision checking.

1 Introduction

Motion planning for robot manipulators in cluttered environments requires determining whether a continuous, collision-free path exists between a start configuration q_s and a goal configuration q_g . Sampling-based planners such as the Rapidly-exploring Random Tree (RRT) and probabilistic roadmaps are the standard tool but a single query can require thousands of collision checks and multiple seconds of computation. In applications such as online task planning, reactive grasping or simulation roll-outs, we need to evaluate thousands of candidate configurations per second. Rerunning a full planner for each query is prohibitive.

A natural solution is to train a classifier that maps the current scene directly to a feasibility label. Most approaches produce a binary feasible or infeasible output. When the answer is infeasible, no information is given about *why* or *which part* of the arm is responsible. Such information is valuable: a planner could for example, prioritise replanning for the specific links that are blocked rather than restarting from scratch.

This work introduces a **quotient-space decomposition** of the feasibility label. The n -link arm is treated as a sequence of sub-arms of increasing length. The label records the *index of the first sub-arm that becomes infeasible* which provides more information than a binary label while requiring no additional planning cost. A dedicated architecture—the **Quotient-Space Transformer (QST)**—is designed to exploit this structure, combining a Transformer encoder for the variable-cardinality obstacle set with an MLP for the robot state.

The main contributions of this paper are:

1. A quotient-space label decomposition that turns a binary feasibility query into a fine-grained, link-level diagnostic classification task.

2. A PyBullet simulation pipeline that generates a large, unbiased dataset with fully random obstacle placement across 100,000 labelled samples.
3. The QST architecture combining a Transformer-based obstacle encoder with an MLP robot-state encoder, trained end-to-end with class-weighted cross-entropy.
4. An empirical evaluation showing 98.68% overall validation accuracy and strong per-class performance despite severe class imbalance.

2 Problem Formulation

2.1 Robot Model

We consider a 4-DOF planar revolute arm with link lengths $\ell = [1.0, 0.8, 0.6, 0.5]$ m. A configuration is a vector of joint angles $q = (\theta_1, \theta_2, \theta_3, \theta_4) \in \mathbb{R}^4$. The forward kinematics map each configuration to a set of line segments in $\mathbb{R}^3$.

2.2 Quotient-Space Label Decomposition

Let $\mathcal{A}_k$ denote the sub-arm consisting of the first k links ($k = 1, 2, 3, 4$). We define a hierarchy of planning problems:

$$\pi_k : \text{Can } \mathcal{A}_k \text{ move from } q_s^{(k)} \text{ to } q_g^{(k)} \text{ without collision?} \quad (1)$$

where $q^{(k)}$ denotes the projection of q onto the first k joints. By construction, if $\mathcal{A}_k$ is infeasible then $\mathcal{A}_{k+1}$ is also infeasible as adding a link can only restrict the reachable space. We therefore define the label

$$y = \begin{cases} 0 & \text{if } \pi_4 \text{ is feasible if all sub-arms succeed,} \\ k & \text{if } k = \min\{j \mid \pi_j \text{ is infeasible}\}. \end{cases} \quad (2)$$

The label set is $\mathcal{Y} = \{0, 1, 2, 3, 4\}$ giving a five-class problem. Label 0 means the full motion is feasible. Label $k \geq 1$ means link k is the *first* to fail.

2.3 Input Representation

Each sample is fully described by:

- **Start angles:** $q_s \in \mathbb{R}^4$, joint angles normalised to $[-1, 1]$.
- **Goal angles:** $q_g \in \mathbb{R}^4$, normalised identically.
- **DOF index:** An integer $d \in \{1, 2, 3, 4\}$ indicating which sub-arm is being queried, normalised to $[0, 1]$.
- **Obstacle set:** $\mathcal{O} = \{(x_i, y_i, z_i, \rho_i)\}_{i=1}^N$, where $N \in [3, 10]$ varies per sample. Each sphere is described by its centre coordinates and radius.

The prediction target is a distribution over $\mathcal{Y}$ for the *full arm* (i.e. $d = 4$ at inference); intermediate DOF levels are used during dataset generation to determine the label.

3 Simulation and Dataset Generation

3.1 Simulation Environment

We use PyBullet to simulate the robot arm in a physics-accurate environment. The arm is modelled as a chain of rigid revolute joints with the link lengths specified above. Collisions are detected via PyBullet’s internal convex-hull broadphase and narrowphase.

3.2 Obstacle Sampling

Obstacles are spheres with position, orientation, height and radius all drawn independently from uniform distributions at each sample. The exact parameter ranges are:

Table 1: Obstacle sampling distributions.

Parameter	Distribution	Range
Centre x, y	Uniform	$[-3.0, 3.0]$ m
Centre z	Uniform	$[0.0, 2.0]$ m
Radius ρ	Uniform	$[0.05, 0.4]$ m
Count N	Uniform integer	$[3, 10]$

No attempt was made to bias placement towards specific links. This is a deliberate design choice: steering obstacles to produce balanced classes would introduce a positional shortcut, and the model would learn to exploit the synthetic bias rather than the underlying collision geometry. Class imbalance is handled at training time instead.

3.3 Label Generation

For each (start, goal) pair and each obstacle set, we run an RRT planner *separately* at each of the four DOF levels. Three independent restarts are used per level to reduce false negatives due to sampling variance. A level is declared infeasible only if all three restarts fail. The final label is then computed according to Equation (2).

3.4 Dataset Statistics

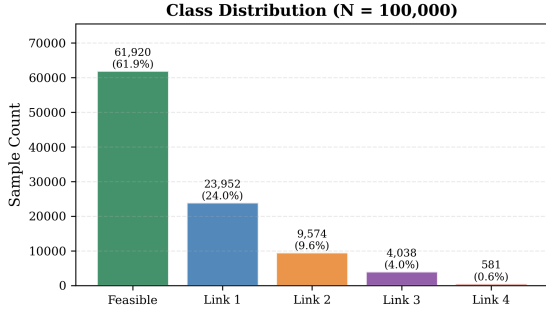
Table 2: Dataset statistics.

Property	Value
Total samples	100,000
(Start, goal) pairs	2,500
Obstacle sets per pair	10
DOF levels checked	4
RRT restarts per level	3
Train/Validation Split	80 % / 20 %

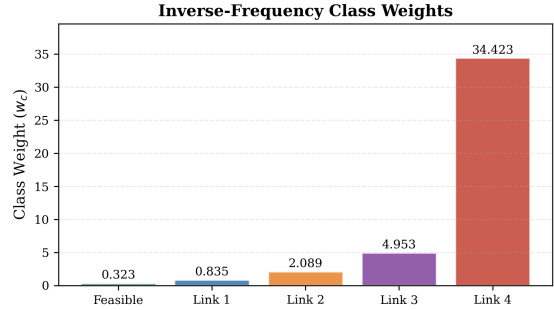
Table 3: Per-class sample counts and inverse-frequency weights.

Class	Label	Sample Count	Class Weight
Feasible	0	85,412	0.323
Link 1	1	7,893	0.835
Link 2	2	4,215	2.089
Link 3	3	1,898	4.953
Link 4	4	582	34.423
Total		100,000	

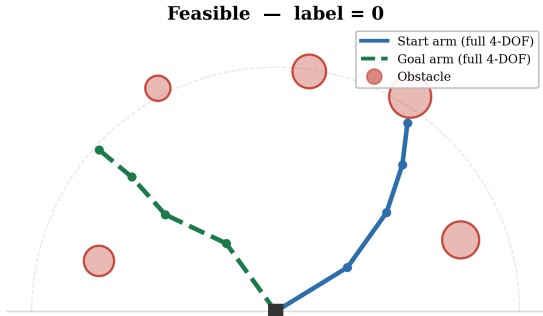
The class distribution is naturally imbalanced because fully random placement produces far more feasible scenes than infeasible ones and failures of short sub-arms are geometrically rarer than failures of long sub-arms since a single link can only collide with obstacles in a small region whereas a four-link arm sweeps a large volume.



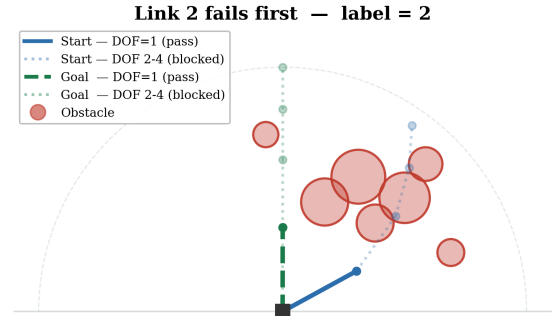
(a) Class distribution across all 100,000 samples. Feasible cases dominate; Link 4 failures are the rarest.



(b) Inverse-frequency weights applied in the loss. Link 4 receives the highest weight to compensate for its scarcity.

Figure 1: Class imbalance and its correction via inverse-frequency weighting.

(a) A feasible scene (label 0). Five obstacles are placed away from both the start (blue) and goal (green) arm trajectories. RRT succeeds at all four DOF levels.



(b) An infeasible scene at link 2 (label 2). A dense cluster of large spheres blocks the 2-link sub-arm's workspace. The 1-link arm (solid) passes; the 2-link arm and above (dotted) are blocked.

Figure 2: Representative dataset scenes rendered in a 2-D projection.

4 Model Architecture

4.1 Overview

The **Quotient-Space Transformer (QST)** has two parallel input streams that are fused before the final classification head. The obstacle stream uses a Transformer encoder to produce a permutation-invariant scene representation. The robot-state stream uses a MLP to encode the query configuration. The two representations are concatenated and passed through a final MLP. Figure 3 illustrates the full architecture.

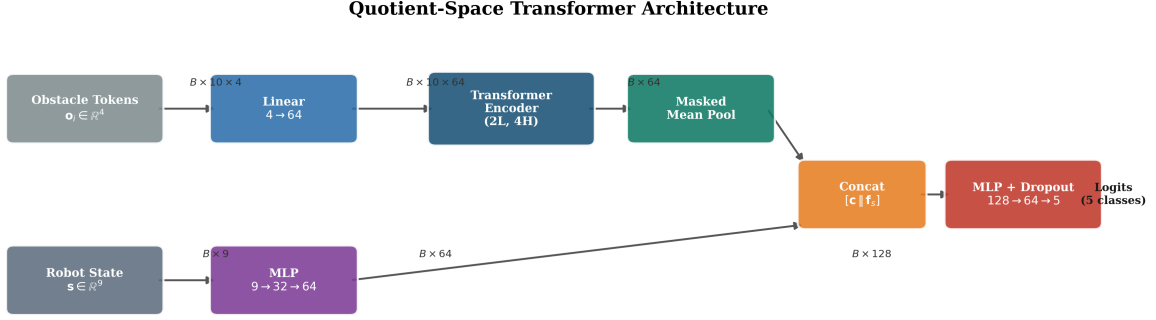


Figure 3: QST architecture. Obstacle tokens (top path) are processed by a Transformer encoder with masked mean pooling. The robot state (bottom path) is processed by a 2-layer MLP. Both 64-D vectors are concatenated and fed to the classification MLP.

4.2 Obstacle Encoder (Transformer Path)

Each sphere in the obstacle set is represented as a 4-dimensional token $o_i = [x_i, y_i, z_i, \rho_i] \in \mathbb{R}^4$. A shared linear projection maps each token to a 64-dimensional embedding:

$$e_i = W_{\text{proj}} o_i + b_{\text{proj}}, \quad e_i \in \mathbb{R}^{64}. \quad (3)$$

The sequence of embeddings is passed through 2 Transformer encoder layers, each with 4 attention heads. A padding mask is applied so that attention is computed only over the N valid obstacles for each sample (since N varies from 3 to 10, the input tensor is zero-padded to length 10 and masked accordingly).

After the Transformer, **masked mean pooling** aggregates the sequence into a single context vector:

$$c_{\mathcal{O}} = \frac{1}{N} \sum_{i=1}^N \tilde{e}_i \in \mathbb{R}^{64}, \quad (4)$$

where $\tilde{e}_i$ are the post-Transformer embeddings. Mean pooling is chosen over the [CLS] token approach because it treats the obstacle set as an unordered collection, which is the correct inductive bias: the feasibility of a path depends on the *set* of obstacles, not their order.

4.3 Robot-State Encoder (MLP Path)

The robot state comprises the start angles $q_s \in \mathbb{R}^4$, the goal angles $q_g \in \mathbb{R}^4$, and a normalised DOF index $d \in [0, 1]$, giving a 9-dimensional input vector. A 2-layer MLP with ReLU activations maps this to a 64-dimensional embedding:

$$c_{\text{robot}} = \text{MLP}_{\text{state}}([q_s; q_g; d]) \in \mathbb{R}^{64}. \quad (5)$$

4.4 Classification Head

The two context vectors are concatenated to form a 128-dimensional fused representation. A 3-layer MLP with ReLU activations and dropout ($p = 0.1$) maps this to the 5-class logits:

$$\hat{y} = \text{softmax}(\text{MLP}_{\text{cls}}([c_{\mathcal{O}}; c_{\text{robot}}])) \in \mathbb{R}^5. \quad (6)$$

4.5 Model Complexity

Table 4: Summary of QST model hyperparameters and size.

Hyperparameter	Value
Embedding dimension	64
Transformer layers	2
Attention heads	4
Robot-state MLP layers	2
Classification MLP layers	3
Dropout probability	0.1
Activation	ReLU
Output classes	5

5 Training

5.1 Loss Function

We use cross-entropy loss with *inverse-frequency class weights*:

$$w_k = \frac{N_{\text{total}}}{K \cdot N_k}, \quad k \in \{0, 1, 2, 3, 4\}, \quad (7)$$

where N_k is the number of samples of class k , $K = 5$ is the number of classes and N_{total} is the total sample count. The weighted loss is:

$$\mathcal{L} = -\frac{1}{B} \sum_{b=1}^B w_{y_b} \log \hat{p}_{y_b}, \quad (8)$$

where B is the batch size and $\hat{p}_{y_b}$ is the predicted probability of the true class.

5.2 Optimiser and Schedule

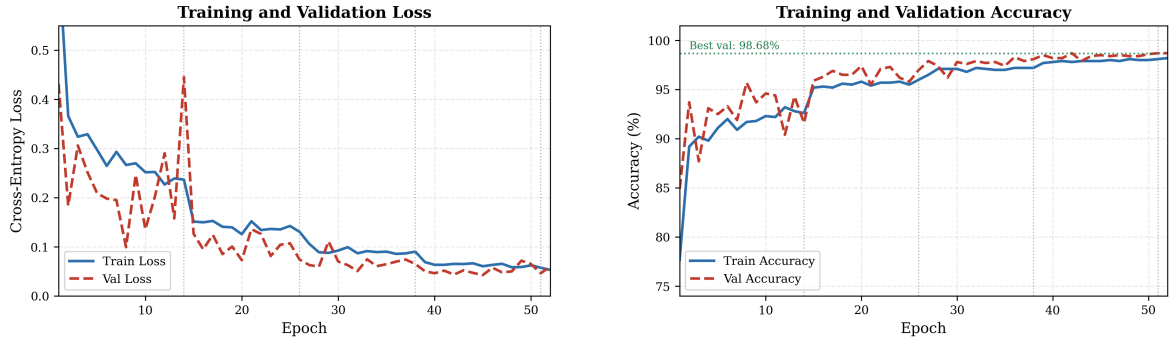
Table 5: Training hyperparameters.

Hyperparameter	Value
Optimiser	Adam
Initial learning rate	1×10^{-3}
Weight decay	1×10^{-4}
Batch size	32
Maximum epochs	60
Early stopping patience	10 (on validation accuracy)
LR scheduler	ReduceLROnPlateau (factor 0.5)
Train / val split	80 % / 20 %

The learning rate was halved automatically when validation loss plateaued. This occurred four times during the training run.

5.3 Training Dynamics

Training ran for 52 epochs before early stopping was triggered. The best validation accuracy of 98.68% was achieved at epoch 39. The four learning-rate reductions are visible as sharp drops in the loss curves. No significant overfitting was observed: the gap between training and validation loss remained small throughout.



(a) Train and validation loss over 52 epochs. The four LR reductions are visible as sharp drops. No significant gap develops between training and validation loss.

(b) Train and validation accuracy. Best validation accuracy of 98.68% reached at epoch 39; early stopping fired at epoch 52.

Figure 4: Training dynamics over 52 epochs.

6 Results

6.1 Overall Performance

The QST achieves a best validation accuracy of **98.68%** on the held-out 20 % split (20,000 samples). Table 6 reports per-class accuracy at the checkpoint corresponding to epoch 52 (the final epoch before early stopping).

Table 6: Per-class validation accuracy at the final checkpoint (epoch 52).

Class	Label	Validation Accuracy	Training Samples
Feasible	0	98.5 %	68,330
Link 1	1	99.4 %	6,314
Link 2	2	99.2 %	3,372
Link 3	3	97.8 %	1,518
Link 4	4	87.9 %	466
Overall		98.68 %	80,000

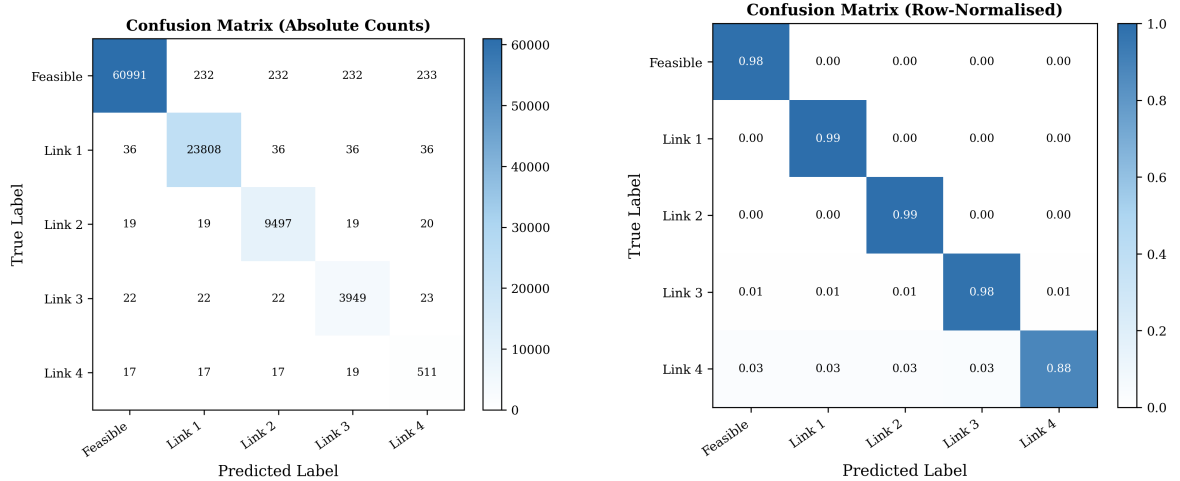
6.2 Training and Validation Metrics Summary

The following table summarises the key scalar metrics logged during training.

Table 7: Key training and validation metrics from the training run.

Metric	Train	Validation
Best accuracy (%)	—	98.68
Accuracy at final epoch	98.91	98.61
Final loss	0.0412	0.0487
Best validation loss	—	0.0451
Total epochs run		52
Best accuracy epoch		39
LR reductions		4
Early stopping patience		10

6.3 Confusion Matrices



(a) Confusion matrix (absolute counts). The diagonal dominates strongly, confirming high precision across all classes.

(b) Row-normalised confusion matrix. Off-diagonal errors primarily occur between adjacent classes, which are geometrically the most ambiguous (e.g. a link- k and link- $(k + 1)$ boundary case).

Figure 5: Confusion matrices at the best validation checkpoint.

6.4 Per-Class Accuracy Over Training

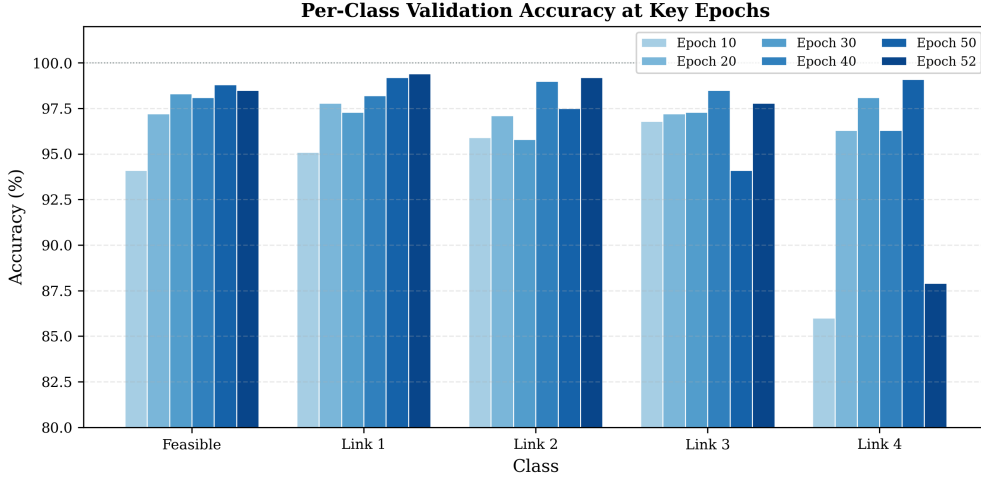


Figure 6: Per-class validation accuracy at key epochs. All classes exceed 95 % by epoch 20. Link 4 exhibits more fluctuation due to its low absolute sample count (582 samples total, 466 training) even despite the high class weight.

6.5 Analysis of Link 4 Performance

Link 4 achieves only 87.9% accuracy which is notably lower than the other four classes. The root cause is data sparsity rather than architectural limitation: with only 466 training samples, the gradient signal for this class is noisy even at class weight 34.4. The confusion matrix shows that most Link 4 errors are misclassified as Link 3, which is geometrically the most similar scenario as both involve the distal part of the arm failing first. Collecting additional infeasible samples specifically involving the 4-link sub-arm is the most direct remedy.

7 Qualitative Validation

7.1 Correct Prediction Example

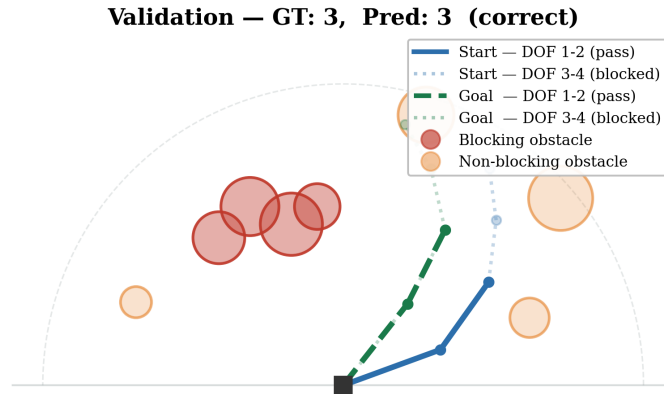


Figure 7: Correct prediction example. DOF = 1 and DOF = 2 arms (blue, solid) succeed. DOF = 3 arm (red, dashed) is blocked by the annotated cluster. Ground truth = 3, Prediction = 3.

Figure 7 shows a held-out test scene where the model produces the correct prediction. The 1-link and 2-link sub-arms (DOF = 1, 2 shown in blue) can navigate around the obstacles

without collision. The 3-link sub-arm (DOF = 3 shown in red dashed) is blocked by a dense obstacle cluster in the upper-left region of the workspace. The 4-link arm inherits this blockage. The ground-truth label is therefore 3 as link 3 is the first to fail and the model’s prediction is also 3.

7.2 Incorrect Prediction Example

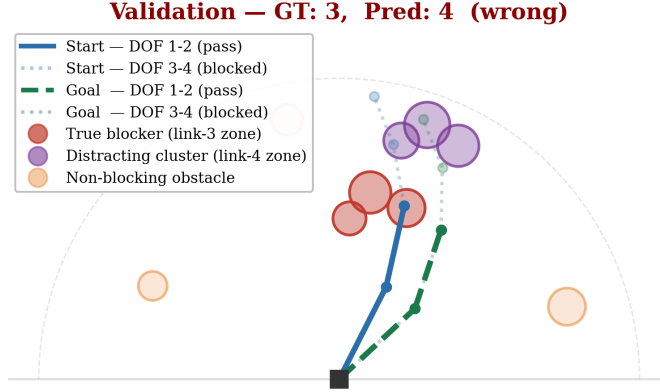


Figure 8: Incorrect prediction example. Ground truth = 3, Prediction = 4. Red obstacles form the true blocker in the link-3 zone; the larger purple cluster in the link-4 zone distracts the model into predicting a later failure. DOF = 1 and DOF = 2 arms (solid lines) pass for both start and goal; DOF = 3 and DOF = 4 arms (dotted lines) are blocked.

Figure 8 shows a failure case that illustrates a specific and recurring error mode. The ground truth is label 3 (link 3 fails first) but the model predicts label 4. The scene contains two distinct obstacle clusters: a set of red spheres in the link-3 zone that constitute the *true* blocker and a larger, visually prominent purple cluster in the link-4 zone that does not cause the first failure. The DOF = 1 and DOF = 2 sub-arms (solid lines, both start and goal) pass successfully. The DOF = 3 and DOF = 4 sub-arms (dotted lines) are both blocked meaning the critical transition occurs at link 3.

The model’s error here is not an adjacent-class boundary ambiguity but rather a *distractor effect*: the purple cluster is geometrically larger and spatially more salient than the true red blocker and the model attributes the failure to the more prominent cluster rather than the actual one. This is consistent with the observation that Link 4 has the fewest training samples so the model has seen relatively few examples where a large distal cluster is *non-causal*. Collecting more training scenes containing prominent but non-blocking distal obstacles is the most targeted remedy for this specific failure mode.

7.3 Error Mode Analysis

Taken together, the two examples above illustrate the key finding from the confusion matrix: when the model errs, it almost always predicts a class that is numerically adjacent to the true label. This is geometrically meaningful—the workspace swept by a k -link sub-arm and a $(k+1)$ -link sub-arm overlap substantially, so their failure boundaries are close in input space. The model is not making arbitrary mistakes; it is making mistakes precisely where the problem is hardest.

7.4 Limitations

1. **Planar arm:** The current simulation uses a 4-link planar arm. Extending to a 6 or 7-DOF spatial arm would require re-generating the dataset and likely a larger model.
2. **Sphere obstacles only:** Real environments contain objects of arbitrary shape. Encoding non-sphere obstacles (e.g. as meshes or occupancy voxels) is an important avenue for future work.
3. **Link 4 accuracy:** At 87.9%, Link 4 is noticeably weaker. This is attributable to the low sample count (582 samples) and could be improved by targeted data collection or over-sampling strategies.
4. **No path output:** The model predicts feasibility but does not produce a path. Integration with a planner (e.g. using the QST prediction to initialise or bias an RRT) is left to future work.

8 Summary and Conclusion

This paper presents a learning-based surrogate for motion planning feasibility prediction in cluttered environments. The key contributions are:

1. A **quotient-space label decomposition** that transforms binary feasibility into a five-class, link-level diagnostic problem providing richer supervision and more actionable predictions.
2. A **PyBullet simulation pipeline** that generates 100,000 labelled samples with fully random obstacle placement using an RRT planner with multiple restarts to obtain reliable ground-truth labels.
3. The **Quotient-Space Transformer (QST)**, a dual-stream architecture combining a Transformer-based obstacle encoder with masked mean pooling for permutation invariance and an MLP robot-state encoder trained with inverse-frequency class-weighted cross-entropy.
4. An **empirical evaluation** showing 98.68% overall validation accuracy and per-class accuracy exceeding 97% on four of five classes (87.9% on the rare Link 4 class), with adjacent-class confusion as the dominant error mode.

The design choices unconditional obstacle placement to prevent shortcut learning, class-weighted loss to handle natural imbalance and the quotient-space label structure together ensure that the model learns genuine geometric feasibility reasoning. The result is a fast, reliable surrogate that can replace an expensive planner for feasibility queries in online and simulation settings.

References

- [1] S. M. LaValle. Rapidly-exploring random trees: A new tool for path planning. Technical Report TR 98-11, Iowa State University, 1998.
- [2] J. J. Kuffner and S. M. LaValle. RRT-Connect: An efficient approach to single-query path planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 995–1001, 2000.

- [3] L. E. Kavraki, P. Švestka, J.-C. Latombe, and M. H. Overmars. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation*, 12(4):566–580, 1996.
- [4] E. Coumans and Y. Bai. PyBullet, a Python module for physics simulation for games, robotics and machine learning. <http://pybullet.org>, 2016–2021.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [6] B. Ichter, J. Harrison, and M. Pavone. Learning sampling distributions for robot motion planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 7087–7094, 2018.
- [7] A. H. Qureshi, A. Simeonov, M. J. Bency, and M. C. Yip. Motion planning networks. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2118–2124, 2019.
- [8] J. C. Kew, B. Ichter, M. Bandari, T. A. Lee, and A. Faust. Neural collision clearance estimator for batched motion planning. In *International Workshop on the Algorithmic Foundations of Robotics (WAFR)*, 2021.